MCMP117 · Week 9 · Course Notes

Fundamentals of Operating Systems

Linux Shell Basics & Filesystem Navigation

From theory to the terminal: where am I, what is here, and how do I move?

Instructor: Asst. Prof. Dr. (Ph.D.) Hasan Oğuz
Head of the Electronical Communication Technology Programme, Vocational School
Istanbul Okan University

What you should be able to do after this week

1. Open a Linux terminal on a lab PC, with WSL on Windows, or in a VirtualBox virtual machine, and create the course lab folder `~/lab`.
2. Explain the difference between the terminal, the shell and the kernel, and describe what happens (fork, exec, wait, system calls) when you type a command.
3. Read the prompt and split a command line into command, options and arguments.
4. Use `whoami`, `hostname`, `date`, `echo`, `uname`, pwd and `ls` with its most useful options, and read every column of `ls -l`.
5. Describe the Linux directory tree (`/`, `/home`, `/etc`, `/bin`, `/usr`, `/var`, `/tmp`, `/dev`, `/proc`).
6. Move around with `cd` using absolute and relative paths and the special names `~ . .. -`; explain why `cd` is a shell builtin.
7. Find help (`--help`, `man`, `help`, `type`) and work efficiently with tab completion, history and keyboard shortcuts.

Contents

1 From the Midterm to the Terminal 1

2 Getting a Linux Terminal 2

3 Terminal, Shell and Kernel 4

4 The Prompt and Your First Commands 5

5 Where Am I? What Is Here? 7

6 The Linux Directory Tree 10

7 Moving Around: cd and Paths 13

8 Getting Help 15

9 Working Smarter: Completion, History, Shortcuts 16

10 Command Reference: Week 9 17

11 Common Errors and What They Mean 17

12 Summary 19

13 Glossary 19

14 Lab Sheet 19

15 Review Questions 21

16 Lab Sheet Solutions 22

17 Looking Ahead: Week 10 24

1 From the Midterm to the Terminal

The first half of the course (Weeks 1–7, examined in the Week 8 midterm) was about *ideas*: the operating system as resource manager and extended machine, the kernel running in kernel mode while everything else runs in user mode, **system calls** as the only door into the kernel (Weeks 2–3), **processes** with their PID, state and PCB, created with `fork()` and `exec()` and scheduled by the CPU scheduler (Weeks 3–5), and finally memory, virtual memory, files and the tree-structured directory (Weeks 6–7).

From this week until Week 14 we work *hands-on* in the Linux shell. You will see the same ideas at work: every command you type becomes a process; every file listing is read from a directory through a system call; the directory tree of Week 7 is the tree you walk with `cd`. Week 10 covers file operations, Week 11 permissions, Week 12 text processing and pipes, Week 13 processes and job control, and Week 14 shell scripting.

About the outputs in these notes. All terminal output was captured on Ubuntu 24.04 LTS with a normal user called `student` on a computer called `okan-lab`, with the locale `LC_ALL=C.UTF-8`. On your machine the user name, host name, dates, times, sizes of system files and process IDs (PIDs) will be different, and with another locale the sort order of `ls` may differ slightly (for example, whether upper-case names come first). The files inside `~/lab` are identical for everybody. Ubuntu also colours the output of `ls` (directories in blue); our captured output has no colours. Every demo is in the `code/` folder as a `demo_*.sh` file with its output in `out_*.txt`.

2 Getting a Linux Terminal

You need a Linux shell for the rest of the semester. There are three good ways to get one; the commands are the same in all three.

2.1 Option 1: the lab PCs

Log in, open the *Terminal* application and continue with Section 2.4. *[The exact room, account and system on the lab PCs will be announced by the instructor.]*

2.2 Option 2: WSL on Windows 10/11

The **Windows Subsystem for Linux** (WSL) runs a real Linux kernel in a lightweight virtual machine managed by Windows, with Ubuntu as the default distribution.

1. Right-click the Start button and choose *Terminal (Admin)* or *Windows PowerShell (Admin)*.
2. Type the following command and press Enter:

   ```
   PS C:\Windows\system32> wsl --install
   ```

   It enables the necessary Windows features, installs WSL 2 and downloads Ubuntu.
3. Restart Windows when asked.
4. Open *Ubuntu* from the Start menu. On the first start it asks for a new *Linux* user name and password (they do not have to match your Windows account). While you type the password *nothing is shown*, not even stars — this is normal on Linux. This user may later run administrator commands with `sudo`.
5. From now on open *Ubuntu* (or type `wsl` in PowerShell) whenever you need the shell.

Useful facts: `wsl -l -v` (in PowerShell) lists the installed distributions and their WSL version; `wsl --install -d Ubuntu-24.04` installs a particular Ubuntu release. Inside Linux, your Windows `C:` drive appears as `/mnt/c`, so your Windows downloads folder is `/mnt/c/Users/<your Windows name>/Downloads`. If installation fails with a message about virtualization, enable it in the BIOS/UEFI setup of your computer (it is called Intel VT-x, AMD-V or SVM).

2.3 Option 3: VirtualBox and Ubuntu

A **virtual machine** applies the OS idea of virtualization to a whole computer: a *hypervisor* (here VirtualBox) gives the guest OS virtual CPUs, memory and a virtual disk, so Ubuntu runs as if it owned the hardware.

1. Download and install VirtualBox, and download the *Ubuntu Desktop 24.04 LTS* ISO image (about 6 GB).
2. In VirtualBox click *New*: give the VM a name, select the ISO image, type Linux / Ubuntu (64-bit).
3. Give it 4096 MB of memory, 2 CPUs and a 25 GB virtual disk (dynamically allocated: the file grows only as needed). If your laptop has only 4 GB of RAM, prefer WSL.
4. Start the VM and follow *Install Ubuntu*. The option “Erase disk and install Ubuntu” erases only the *virtual* disk, a file on your computer — not your Windows disk. (VirtualBox 7 can also do an “unattended installation” that fills in the user details for you.)

5. Log in and open a terminal with `Ctrl+Alt+T` (or *Activities* → *Terminal*).

Mac users. The macOS Terminal (with the shell `zsh`) understands most commands of this week, but many options of the BSD tools on macOS differ from the GNU tools on Linux. For the coming weeks use a Linux VM (on Apple Silicon Macs, UTM is a free option; VirtualBox support there is limited).

2.4 Creating the lab folder

All practical weeks use the same practice folder `~/lab`. It is created by the script `mcmp117_lab_setup.sh`, which you find in the `code/` folder of every Linux week and in the course's GitHub repository. Run it *once* with `bash`; it always builds the folder `lab` in your home directory, wherever the script itself is:

```
# Ubuntu / VM: the script is in your Downloads folder
student@okan-lab:~$ bash ~/Downloads/mcmp117_lab_setup.sh
# WSL: the script is in your Windows Downloads folder (use your Windows name)
student@okan-lab:~$ bash /mnt/c/Users/ayse/Downloads/mcmp117_lab_setup.sh
```

With the script in the home directory, the real output looks like this (`ls` lists a directory; Section 5):

```
student@okan-lab:~$ cd
student@okan-lab:~$ pwd
/home/student
student@okan-lab:~$ ls
lab  mcmp117_lab_setup.sh
student@okan-lab:~$ bash mcmp117_lab_setup.sh
Lab folder ready: /home/student/lab
README.txt  backup  data  docs  logs  photos  projects  scripts  src
student@okan-lab:~$ ls
lab  mcmp117_lab_setup.sh
student@okan-lab:~$ ls lab
README.txt  backup  data  docs  logs  photos  projects  scripts  src
```

The script contains no randomness: everybody gets exactly the same files as these notes. It is safe to run again — it deletes and re-creates *only* the folder `lab` — so whenever your lab folder gets into a mess, simply run it again. For the same reason, never keep your own work inside `~/lab`.

> **If the script fails**
>
> If you see an error such as `line 2: $'\r': command not found`, the file got Windows line endings (for example after being opened and saved in Notepad). Download it again, or repair it with `sed -i 's/\r$//' mcmp117_lab_setup.sh`. If you see `No such file or directory`, check the path with `ls` first (Section 11).

3 Terminal, Shell and Kernel

3.1 Three programs, three jobs

People often use "terminal", "shell", "command line" and "Linux" as if they meant the same thing. They are different layers:

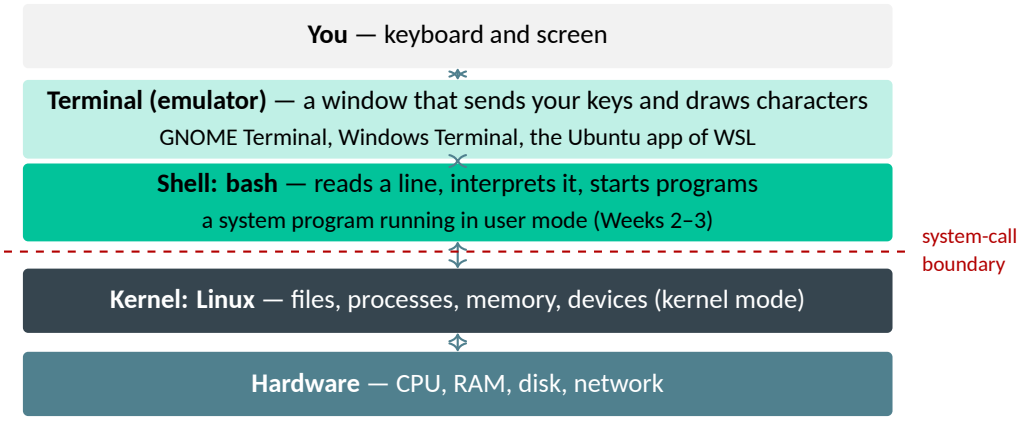


Figure 1: The layers between you and the hardware. Everything above the dashed line runs in user mode.

- The **terminal** only handles input and output. It does not understand commands. (Historically a terminal was a physical device — a teletype or a video terminal such as the VT100; today it is a program that emulates one.)
- The **shell** is the *command interpreter* of Week 3. Linux offers several: `bash` (the default on Ubuntu), `zsh`, `sh`, `fish`. As Week 2 stressed, the shell is *not* part of the kernel; it is an ordinary system program.
- The **kernel** does the real work for every command, and is reached only through system calls.
- Strictly, "Linux" is only the kernel; Ubuntu is a *distribution*: the Linux kernel plus the GNU tools (`bash`, `ls`, …) and thousands of other programs.

3.2 What happens when you type a command

When you type `ls` and press Enter, `bash` works through the *fork–exec–wait* pattern of Week 3:

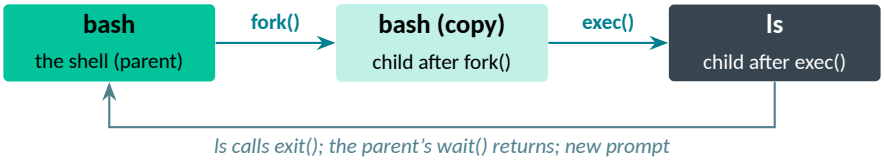


Figure 2: The shell creates a new process for every external command.

1. **Read:** `bash` reads the line `ls` from the terminal.
2. **Find:** `ls` is not a builtin, so `bash` searches the directories listed in the variable `$PATH` and finds the program file `/usr/bin/ls`.
3. **fork():** `bash` creates a child process, a copy of itself, with a new PID.
4. **exec():** the child replaces its program with `/usr/bin/ls`, which runs, prints and exits.
5. **wait():** the parent shell waits for the child to finish and then prints the next prompt.

Some commands, called **builtins**, are executed by `bash` itself without creating a process: `cd`, `pwd`, `echo`, `history`, `type` and others. Section 7.4 explains why `cd` *has* to be a builtin.

3.3 Commands become system calls

In Week 2 we used `strace` to list the system calls of a C program. It works for any command. Here we trace only three kinds of system call, and `> /dev/null` throws away the normal output of `ls` so that only the report of `strace` remains (redirection is the topic of Week 12):

```
student@okan-lab:~/lab$ strace -e trace=execve,getdents64,write ls docs > /dev/null
execve("/usr/bin/ls", ["ls", "docs"], 0x7ffc5b53bce8 /* 13 vars */) = 0
getdents64(3, 0x55fd38daa8e0 /* 9 entries */, 32768) = 288
getdents64(3, 0x55fd38daa8e0 /* 0 entries */, 32768) = 0
write(1, "final report.txt\nlinux_quotes.tx"..., 86) = 86
+++ exited with 0 +++
```

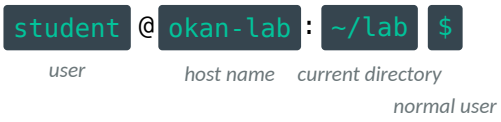
- `execve`: the child process becomes `/usr/bin/ls` with the argument list `["ls", "docs"]` — step 4 above.
- `getdents64` ("get directory entries") reads the directory. It found 9 entries: the 6 visible files, the hidden file `.settings`, and the two entries `.` and `..` that every directory contains. A second call returns 0 entries: the end of the directory. Recall from Week 7 that a directory is a file containing (name, inode number) pairs; this is the system call that reads those pairs.
- `write(1, …, 86)` writes 86 bytes to file descriptor 1, standard output.

`ls` cannot read the disk on its own; like every user-mode program it must ask the kernel (dual mode, Week 2). Without the filter, `ls` makes several dozen system calls (loading libraries, reading locale files, mapping memory). If `strace` is missing: `sudo apt install strace`.

4 The Prompt and Your First Commands

4.1 Reading the prompt

When the shell is ready for a command it prints a **prompt**. The default Ubuntu prompt has the form `user@host:directory$`:



The tilde `~` is short for your **home directory**, here `/home/student`. A prompt ending in `#` instead of `$` means you are `root`, the administrator — be careful then. In these notes, the text after the prompt is what you type; the lines without a prompt are the output.

4.2 Anatomy of a command

A command line is split at spaces into *words*:

`command` `-options arguments` e.g. `ls -l -a docs`

- The first word is the **command**: a program file or a builtin.

- **Options** change *how* the command works. Short options are one letter after a dash and can be combined: `ls -l -a` = `ls -la` = `ls -al`. Long options start with two dashes and are spelled out: `ls --all`. Some options take a value: `head -n 4`.
- **Arguments** say *what* the command works on: files, directories, text.

Linux is *case-sensitive*: `ls` and LS are different words, and so are `docs` and `Docs`. The spaces matter too: `ls-l` is one word, a command that does not exist.

Example	Command	Options	Arguments
`pwd`	`pwd`	—	—
`ls -la docs`	`ls`	`-l -a` (combined)	`docs`
`ls --all docs`	`ls`	`--all` (long form)	`docs`
`cd ../src`	`cd`	—	`../src`
`head -n 4 file`	`head`	`-n` with value 4	`file`

4.3 First commands

```
student@okan-lab:~$ whoami
student
student@okan-lab:~$ hostname
okan-lab
student@okan-lab:~$ date
Mon Oct 12 08:49:13 UTC 2026
student@okan-lab:~$ echo Hello from the shell
Hello from the shell
student@okan-lab:~$ echo My shell is $SHELL and my home is ~
My shell is /bin/bash and my home is /home/student
student@okan-lab:~$ uname
Linux
student@okan-lab:~$ uname -r
6.18.44-fc-v114
student@okan-lab:~$ uname -a
Linux okan-lab 6.18.44-fc-v114 #1 SMP PREEMPT_DYNAMIC @0 x86_64 x86_64 x86_64 GNU/Linux
```

- `whoami` prints your user name; `hostname` the name of the computer; `date` the date and time (UTC on our test machine; yours shows local time).
- `echo` prints its arguments. Notice that `echo` never saw `$SHELL` or `~`: the *shell* replaced them by `/bin/bash` and `/home/student` before starting the command. This *expansion* by the shell is a key idea for Weeks 10 (wildcards) and 14 (variables).
- `uname` prints the kernel name; `-r` the kernel release; `-a` everything (kernel name, host name, release, build information, machine architecture `x86_64`). On a normal Ubuntu 24.04 PC the release looks like `6.8.0-…-generic`; on WSL it ends in `microsoft-standard-WSL2`.
- `clear` (or `Ctrl+L`) clears the screen.

5 Where Am I? What Is Here?

Every process — and therefore every shell — has a **current working directory**. Relative paths (Section 7) start there. `pwd` (*print working directory*) shows it; `ls` (*list*) shows what it contains.

5.1 pwd and ls

```
student@okan-lab:~$ cd ~/lab
student@okan-lab:~/lab$ pwd
/home/student/lab
student@okan-lab:~/lab$ ls
README.txt  backup  data  docs  logs  photos  projects  scripts  src
student@okan-lab:~/lab$ ls -F
README.txt  backup/  data/  docs/  logs/  photos/  projects/  scripts/  src/
student@okan-lab:~/lab$ ls docs
'final report.txt'   notes_01.txt   notes_03.txt
 linux_quotes.txt    notes_02.txt   syllabus.txt
student@okan-lab:~/lab$ ls -a docs
 .    .settings           linux_quotes.txt   notes_02.txt   syllabus.txt
 ..  'final report.txt'   notes_01.txt       notes_03.txt
```

- `pwd` always prints an *absolute* path, starting at the root `/`.
- `ls` without arguments lists the current directory; `ls docs` lists another directory. The names are sorted and arranged in columns.
- `-F` adds a mark to the type: `/` for directories (also `*` for executables and `@` for symbolic links).
- `-a` (*all*) also shows **hidden** names, which are simply names starting with a dot: here `.settings`. Configuration files are usually hidden, for example `~/.bashrc`. You also see `.` (the directory itself) and `..` (its parent) — exactly the special entries of Week 7. `-A` shows hidden files without `.` and `..`.
- The name `'final report.txt'` is shown in quotes because it contains a space. The quotes are *not* part of the name; GNU `ls` adds them so that you can see where the name begins and ends.

5.2 The long listing: ls -l

```
student@okan-lab:~/lab$ ls -l docs
total 24
-rw-rw-r-- 1 student student  53 Oct 12 08:49 'final report.txt'
-rw-rw-r-- 1 student student 302 Oct 12 08:49  linux_quotes.txt
-rw-rw-r-- 1 student student  16 Oct 12 08:49  notes_01.txt
-rw-rw-r-- 1 student student  16 Oct 12 08:49  notes_02.txt
-rw-rw-r-- 1 student student  16 Oct 12 08:49  notes_03.txt
-rw-rw-r-- 1 student student 578 Oct 12 08:49  syllabus.txt
```

Each line of `ls -l` describes one file. The columns are:

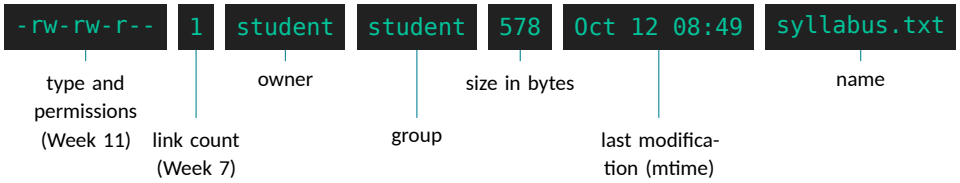


Figure 3: The seven columns of a line of `ls -l`.

1. **Type and permissions.** The first character is the type: `-` regular file, `d` directory, `l` symbolic link, `c` character device. The next nine characters are the read/write/execute permissions for the owner, the group and all others; Week 11 is devoted to them.
2. **Link count.** For a file: the number of hard links (names) pointing to its inode (Week 7). For a directory: $2 +$ the number of its subdirectories (see below).
3. **Owner** and **group** of the file.
4. **Size** in bytes. A directory shows the size of its own table of entries, usually one block of 4096 bytes.
5. **Modification time** (mtime): the last change of the *contents*. Files older than six months show the year instead of the time (`Apr 22 2024` in Section 6).
6. **Name.**

> **ls -l shows the inode**
>
> Apart from the name, every column of `ls -l` is a field of the file's *inode* (Week 7): type, permissions, link count, owner, group, size and times. The name comes from the directory entry. Week 10 shows the complete inode with `stat`.

Directory link counts. Why does `projects` have link count 4 and `docs` only 2? A directory is named by its entry in the parent directory, by its own entry `.`, and by the entry `..` inside each of its subdirectories. Hence

$$\text{link count of a directory} = 2 + s,$$

where s is the number of subdirectories: `projects` has $s = 2$ (`alpha` and `beta`), so $2 + 2 = 4$; `docs` has $s = 0$, so 2. (Checked with `code/check_numbers.py`.)

The `total` line. The first line, `total 24`, is the disk space used by the listed files in units of 1 KiB. Even a 16-byte file occupies a whole file-system block of 4 KiB, so six small files use $6 \times 4 = 24$ KiB (the block idea of Week 7). In `~/lab` itself, `ls -l` shows `total 36`: nine entries, $9 \times 4 = 36$.

5.3 Human-readable sizes: ls -lh

```
student@okan-lab:~/lab$ ls -lh logs data
data:
total 12K
-rw-rw-r-- 1 student student  227 Oct 12 08:49 cities.txt
-rw-rw-r-- 1 student student   80 Oct 12 08:49 numbers.txt
-rw-rw-r-- 1 student student 1.2K Oct 12 08:49 students.csv
```

```
logs:
total 32K
-rw-rw-r-- 1 student student  24K Oct 12 08:49 access.log
```

With `-h` the sizes are printed with the suffixes K, M, G, which here mean powers of 1024: $1\,\text{K} = 2^{10} = 1024$ bytes, $1\,\text{M} = 2^{20}$ bytes. `ls` rounds *up*, and shows one decimal for values below 10:

From bytes to ls -h

$$\begin{aligned} \texttt{access.log}: &\quad 23\,604/1024 \approx 23.05 \longrightarrow \texttt{24K}\ (\text{rounded up}) \\ \texttt{system.log}: &\quad 5\,207/1024 \approx 5.085 \longrightarrow \texttt{5.1K} \\ \texttt{students.csv}: &\quad 1\,208/1024 \approx 1.180 \longrightarrow \texttt{1.2K} \end{aligned}$$

Files smaller than 1024 bytes (such as `cities.txt`, 227 bytes) are shown in bytes. The exact byte counts come from `ls -l` (Lab Sheet, Exercise B7).

5.4 Whole subtrees: ls -R and tree

`ls -R` (*recursive*) lists a directory and then every subdirectory below it, one block per directory. The program `tree` draws the same information as a picture — the tree of Week 7. (`tree` may need to be installed: `sudo apt install tree`.)

```
student@okan-lab:~/lab$ ls -R projects
projects:
alpha  beta

projects/alpha:
README.md  docs  src

projects/alpha/docs:
design.txt

projects/alpha/src:
main.c
```

… (the same for `projects/beta`; full output in `code/out_tree.txt`)

```
student@okan-lab:~/lab$ tree projects
projects
├── alpha
│   ├── README.md
│   ├── docs
│   │   └── design.txt
│   └── src
│       └── main.c
└── beta
    ├── README.md
    ├── docs
    │   └── design.txt
    └── src
        └── main.c

7 directories, 6 files
```

`tree` counts `projects` itself, hence "7 directories". **Careful:** `ls -R /` would list the entire file system — hundreds of thousands of lines. Press `Ctrl+C` to stop a command that runs too long.

5.5 The most useful ls options

Option	Meaning	Example
`-l`	long listing (Figure 3)	`ls -l docs`
`-a / -A`	all names, including hidden ones (-A: without `.` and `..`)	`ls -a ~`
`-h`	human-readable sizes (with -l)	`ls -lh logs`
`-R`	recursive: also every subdirectory	`ls -R projects`
`-F`	mark type: `/` directory, `*` executable, `@` link	`ls -F`
`-d`	the directory itself, not its contents	`ls -ld projects`
`-S`	sort by size, largest first	`ls -lS docs`
`-t`	sort by modification time, newest first	`ls -lt`
`-r`	reverse the sort order	`ls -ltr`
`-1`	one name per line	`ls -1 photos`
`-i`	show inode numbers (Week 7)	`ls -i`

Ubuntu's default `~/.bashrc` also defines the shortcuts (*aliases*) `ll` = `ls -alF`, `la` = `ls -A` and `l` = `ls -CF`.

6 The Linux Directory Tree

Linux has *one* tree with *one* root, `/`. There are no drive letters: other disks and USB sticks are *mounted* into the tree (typically under `/media` or `/mnt`). The **Filesystem Hierarchy Standard** (FHS) says which directory holds what, so that all distributions look alike.

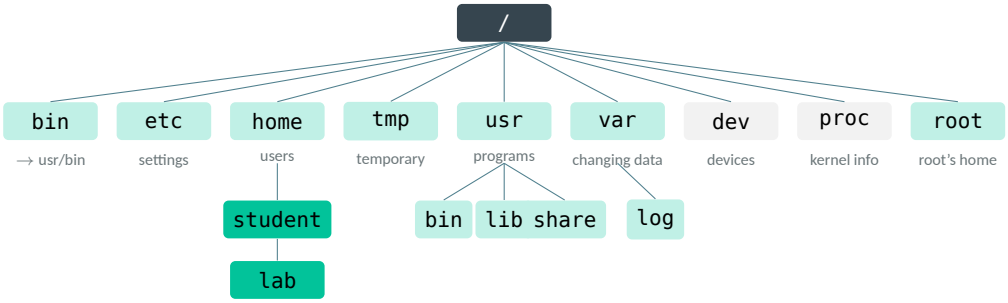


Figure 4: The top of the Linux directory tree. Grey: virtual file systems generated by the kernel.

Directory	Contents	Try
`/`	the root of the whole tree	`ls /`
`/home`	one home directory per user: `/home/student`	`ls ~`
`/root`	home directory of the administrator `root` (not the same as `/`)	`---`
`/etc`	system-wide configuration, mostly plain text files	`head -n 4 /etc/os-release`
`/usr`	installed software: `/usr/bin` programs, `/usr/lib` libraries, `/usr/share` data	`ls -l /usr/bin/ls`
`/bin, /sbin, /lib`	on Ubuntu 24.04 symbolic links to `/usr/bin`, `/usr/sbin`, `/usr/lib`	`ls -ld /bin`
`/var`	variable data that grows: logs (`/var/log`), caches, mail, spool	`ls /var`
`/tmp`	temporary files; everyone may write; often emptied at boot	`ls -ld /tmp`
`/dev`	device files: disks, terminals, `/dev/null`	`ls -l /dev/null`
`/proc`	virtual: information about processes and the kernel, generated on the fly	`head /proc/meminfo`
`/media, /mnt`	mount points for removable and other disks	`---`
`/boot`	the kernel image and boot loader files	`---`

6.1 A tour

In the listing of `/` below, two entries that exist only on our test machine have been left out. The directories ending in `.usr-is-merged` are a harmless leftover of Ubuntu's move to a "merged `/usr`".

```
student@okan-lab:~$ type ls
ls is /usr/bin/ls
student@okan-lab:~$ cd /
student@okan-lab:/$ ls
bin                etc               lib64       opt   sbin                tmp
bin.usr-is-merged  home              lost+found  proc  sbin.usr-is-merged  usr
boot               lib               media       root  srv                 var
dev                lib.usr-is-merged  mnt         run   sys
student@okan-lab:/$ ls -ld /bin /sbin /lib
lrwxrwxrwx 1 root root 7 Apr 22  2024 /bin -> usr/bin
lrwxrwxrwx 1 root root 7 Apr 22  2024 /lib -> usr/lib
lrwxrwxrwx 1 root root 8 Apr 22  2024 /sbin -> usr/sbin
student@okan-lab:/$ ls -l /usr/bin/ls
```

```
-rwxr-xr-x 1 root root 142312 Jan 23  2026 /usr/bin/ls
student@okan-lab:/$ head -n 4 /etc/os-release
PRETTY_NAME="Ubuntu 24.04.4 LTS"
NAME="Ubuntu"
VERSION_ID="24.04"
VERSION="24.04.4 LTS (Noble Numbat)"
student@okan-lab:/$ ls /var
backups  cache  lib  local  lock  log  mail  opt  run  spool  tmp
student@okan-lab:/$ ls -ld /tmp
drwxrwxrwt 16 root root 4096 Oct 12 08:49 /tmp
```

- `type ls` reveals that `ls` is simply a program file, `/usr/bin/ls` (142 312 bytes). Most commands are files like this.
- `/bin`, `/sbin` and `/lib` are **symbolic links** (type `l`, arrow `->`; Week 7) to their counterparts in `/usr`. Both names lead to the same programs.
- `/etc/os-release` names the distribution. UNIX keeps configuration as editable text; Week 11 reads `/etc/passwd` and `/etc/group`.
- `/tmp` has permission string `drwxrwxrwt`: everyone may create files there, and the final `t` (the *sticky bit*, Week 11) stops users from deleting each other's files.

6.2 /dev and /proc: everything is a file

```
student@okan-lab:/$ ls -l /dev/null /dev/tty /dev/zero
crw-rw-rw- 1 root root 1, 3 Oct  9 23:24 /dev/null
crw-rw-rw- 1 root root 5, 0 Oct  9 23:24 /dev/tty
crw-rw-rw- 1 root root 1, 5 Oct  9 23:24 /dev/zero
student@okan-lab:/$ head -n 3 /proc/meminfo
MemTotal:        8222320 kB
MemFree:         7069620 kB
MemAvailable:    7539020 kB
student@okan-lab:/$ echo $$
18432
student@okan-lab:/$ head -n 7 /proc/$$/status
Name:   bash
Umask:  0002
State:  S (sleeping)
Tgid:   18432
Ngid:   0
Pid:    18432
PPid:   18377
```

- In `/dev`, the `c` marks *character devices*. Instead of a size, `ls -l` shows two driver numbers (major, minor), e.g. `1, 3`. `/dev/null` discards everything written to it (we used it with `strace`); `/dev/zero` produces zero bytes; `/dev/tty` is your terminal. Programs use these devices with the same `open`/`read`/`write` system calls as ordinary files (Week 2, I/O).
- `/proc` is not stored on any disk. When you read one of its files, the kernel writes the text on the fly. `/proc/meminfo` shows that our test machine has about 8 GB of RAM (8 222 320 kB).
- `$$` is replaced by the shell with its own PID. `/proc/<PID>/status` shows part of that process's PCB (Week 3): its name (`bash`), state `S` (sleeping — it is waiting for `head` to finish), PID and parent PID

(PPid). Week 13 returns to `/proc` and `ps`.

7 Moving Around: cd and Paths

7.1 Absolute and relative paths

As in Week 7, a file or directory can be named in two ways:

- An **absolute path** starts with `/` and walks down from the root. It means the same thing from anywhere: `/home/student/lab/projects/beta/docs`.
- A **relative path** does not start with `/`; it walks from the current directory. `..` means "one level up".

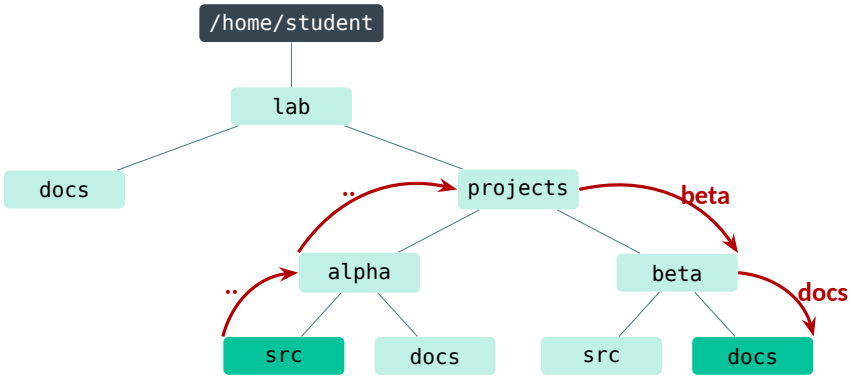


Figure 5: From `projects/alpha/src` to `projects/beta/docs`: `../../beta/docs` (up, up, down, down).

7.2 cd in practice

```
student@okan-lab:~$ cd ~/lab
student@okan-lab:~/lab$ pwd
/home/student/lab
student@okan-lab:~/lab$ cd projects/alpha/src
student@okan-lab:~/lab/projects/alpha/src$ pwd
/home/student/lab/projects/alpha/src
student@okan-lab:~/lab/projects/alpha/src$ cd ..
student@okan-lab:~/lab/projects/alpha$ pwd
/home/student/lab/projects/alpha
student@okan-lab:~/lab/projects/alpha$ cd ../../docs
student@okan-lab:~/lab/docs$ pwd
/home/student/lab/docs
student@okan-lab:~/lab/docs$ cd /home/student/lab/projects/beta
student@okan-lab:~/lab/projects/beta$ pwd
/home/student/lab/projects/beta
student@okan-lab:~/lab/projects/beta$ cd -
/home/student/lab/docs
student@okan-lab:~/lab/docs$ cd -
/home/student/lab/projects/beta
student@okan-lab:~/lab/projects/beta$ cd
student@okan-lab:~$ pwd
/home/student
```

What to observe:

- The prompt shows the new current directory after every `cd`.
- `cd` prints *nothing* when it succeeds ("no news is good news" is a UNIX rule). The only exception is `cd -`, which prints the directory it jumps back to.
- `cd ../../docs` from `~/lab/projects/alpha` goes up twice (to `~/lab`) and then down into `docs`.
- The absolute path `/home/student/lab/projects/beta` works no matter where you are.
- `cd` without an argument takes you home; `cd /` to the root.

7.3 Five special names

Name	Meaning	Example	Result from `~/lab/projects/alpha`
`/`	the root of the tree	`cd /`	`/`
`~`	your home directory	`cd ~/lab`	`/home/student/lab`
`.`	the current directory	`ls .`	lists `alpha` (same as `ls`)
`..`	the parent directory	`cd ../beta`	`~/lab/projects/beta`
`-`	the previous directory (`cd` only)	`cd -`	back to where you were

`.` and `..` are real directory entries (`ls -a` shows them); `~` and `-` are shortcuts of the shell: `~` is replaced by the value of `$HOME`, and `cd -` uses the variable `$OLDPWD`, as `help cd` explains (Section 8). The `..` of `/` is `/` itself: you cannot go above the root. The name `.` becomes important in Weeks 11 and 14: `./script.sh` runs a script from the current directory.

7.4 Why cd must be a shell builtin

```
student@okan-lab:~/lab$ /bin/pwd
/home/student/lab
student@okan-lab:~/lab$ strace -e trace=getcwd /bin/pwd
getcwd("/home/student/lab", 4096)       = 18
/home/student/lab
+++ exited with 0 +++
student@okan-lab:~/lab$ strace -e trace=chdir bash -c 'cd /tmp'
chdir("/tmp")                           = 0
+++ exited with 0 +++
student@okan-lab:~/lab$ pwd
/home/student/lab
```

- `/bin/pwd` is a separate program (not the builtin `pwd`); it asks the kernel for the current directory with the system call `getcwd`. It prints `/home/student/lab`: a child inherits the current directory of its parent.
- `bash -c 'cd /tmp'` starts a *child* shell that runs `cd /tmp`. The kernel call `chdir("/tmp")` succeeds — but afterwards our own shell is still in `~/lab`.

The current directory belongs to the process

The current working directory is part of the state the kernel keeps for each process, next to its PCB (Week 3). A process can change only its *own* current directory, never its parent's. If `cd` were a program in `/usr/bin`, the shell would fork a child, the child would change *its* directory and exit — and the shell would stay where it was. Therefore `cd` is built into the shell: `bash` calls `chdir()` on itself. `type cd` confirms: "cd is a shell builtin". The same argument applies to `exit` and to commands that set shell variables.

8 Getting Help

Nobody remembers every option. Know where to look:

`command --help` A summary of the options, printed by most programs.

`man command` The full *manual page*, divided into sections such as NAME, SYNOPSIS, DESCRIPTION, OPTIONS and EXAMPLES. Move with the arrow keys or Space (next page), search with `/word` and Enter (n for the next match), quit with `q`. `man man` explains `man` itself.

`help command` Help for shell builtins (`cd`, `pwd`, `history`), which have no program file and often no manual page of their own.

`type command` Tells whether a command is a builtin, an alias or a program file (and where).

`whatis` / `apropos` `whatis ls` prints the one-line description from the manual; `apropos directory` searches all manual descriptions for a keyword — useful when you do not know the command's name.

`tldr command` (optional, `sudo apt install tldr`) community-written short examples of common uses.

```
student@okan-lab:~$ ls --help | head -n 12
Usage: ls [OPTION]... [FILE]...
List information about the FILEs (the current directory by default).
Sort entries alphabetically if none of -cftuvSUX nor --sort is specified.

Mandatory arguments to long options are mandatory for short options too.
  -a, --all                  do not ignore entries starting with .
  -A, --almost-all           do not list implied . and ..
```

```
student@okan-lab:~$ type cd pwd ls
cd is a shell builtin
pwd is a shell builtin
ls is /usr/bin/ls
student@okan-lab:~$ help cd | head -n 5
cd: cd [-L|[-P [-e]] [-@]] [dir]
    Change the shell working directory.

    Change the current directory to DIR.  The default DIR is the value of the
    HOME shell variable. If DIR is "-", it is converted to $OLDPWD.
```

Reading a SYNOPSIS: `ls [OPTION]... [FILE]...` means "any number of options, then any number of files, all optional" — square brackets mark optional parts and `...` means "may be repeated". The output

of `ls --help` was shortened with `| head -n 12` (only the first 12 lines; pipes are the topic of Week 12).

Our test machine is a *minimised* Ubuntu without manual pages, which is why these notes show `--help` output instead of `man` output; normal installations, WSL and the lab PCs have them (`sudo unminimize` restores them on a minimised system).

9 Working Smarter: Completion, History, Shortcuts

9.1 Tab completion

Press Tab after the first letters of a command, file or directory name and the shell completes it. In the session below, the user typed `cd pro` Tab `al` Tab and the shell completed it to `cd projects/alpha/`; `ls ~/lab/docs/sy` Tab became `ls ~/lab/docs/syllabus.txt` (with a space added after the complete name). If the beginning is ambiguous (`notes_0` could be three files), nothing seems to happen: press Tab *twice* to see the list of candidates. Tab completion saves typing *and* prevents typing errors.

9.2 History

```
student@okan-lab:~$ cd ~/lab
student@okan-lab:~/lab$ cd projects/alpha/
student@okan-lab:~/lab/projects/alpha$ pwd
/home/student/lab/projects/alpha
student@okan-lab:~/lab/projects/alpha$ cd
student@okan-lab:~$ ls ~/lab/docs/syllabus.txt
/home/student/lab/docs/syllabus.txt
student@okan-lab:~$ whoami
student
student@okan-lab:~$ !!
whoami
student
student@okan-lab:~$ history
    1  cd ~/lab
    2  cd projects/alpha/
    3  pwd
    4  cd
    5  ls ~/lab/docs/syllabus.txt
    6  whoami
    7  whoami
    8  history
student@okan-lab:~$ !5
ls ~/lab/docs/syllabus.txt
/home/student/lab/docs/syllabus.txt
```

The shell remembers your commands. `history` prints them with numbers; `!!` repeats the last command and `!5` repeats command number 5 (the shell first prints the command it is going to run). The arrow keys ↑/↓ walk through the history, and `Ctrl`+R searches it backwards as you type. The history is saved in the file `~/.bash_history` when the shell exits.

9.3 Keyboard shortcuts

Key	Effect
`Tab / Tab Tab`	complete a name / list the candidates
`↑ / ↓`	previous / next command from the history
`Ctrl+R`	search the history backwards
`Ctrl+C`	interrupt (stop) the running command — it sends the signal SIGINT (Week 13)
`Ctrl+L`	clear the screen (like `clear`)
`Ctrl+A / Ctrl+E`	jump to the beginning / end of the line
`Ctrl+D`	end of input; at an empty prompt it closes the shell (like `exit`)

Copy and paste in most terminal windows use `Ctrl+Shift+C` and `Ctrl+Shift+V`, because `Ctrl+C` already means "interrupt".

10 Command Reference: Week 9

Command	Purpose
`whoami`	print your user name
`hostname`	print the computer's name
`date`	print the date and time
`echo text`	print its arguments (after the shell has expanded them)
`uname [-r|-a]`	kernel name / release / all information
`clear`	clear the screen (`Ctrl+L`)
`pwd`	print the current working directory (absolute path)
`ls [-laAhRFdStr1i] [dir...]`	list directory contents (Section 5)
`tree [dir]`	draw a directory tree
`cd [dir]`	change directory; `~ . .. - /`; no argument: home
`cat file`	print a small file (Week 10 covers `cat`, `less`, `head`, `tail`)
`head -n N file`	print the first N lines of a file
`type cmd`	builtin, alias or program file?
`cmd --help`	short help of a program
`man cmd`	manual page (`q` quits, `/word` searches)
`help cmd`	help for a shell builtin
`whatis cmd / apropos word`	one-line description / search the manuals
`history, !!, !n`	list / repeat previous commands
`strace cmd`	show the system calls of a command (Week 2)
`bash file.sh`	run a shell script (e.g. the lab setup script)

11 Common Errors and What They Mean

Error messages are precise; read them. All of the following were produced in `~/lab`:

```
student@okan-lab:~/lab$ LS
bash: LS: command not found
student@okan-lab:~/lab$ sl
bash: sl: command not found
student@okan-lab:~/lab$ ls -z
ls: invalid option -- 'z'
Try 'ls --help' for more information.
student@okan-lab:~/lab$ ls Docs
ls: cannot access 'Docs': No such file or directory
student@okan-lab:~/lab$ cd nowhere
bash: cd: nowhere: No such file or directory
student@okan-lab:~/lab$ cd README.txt
bash: cd: README.txt: Not a directory
student@okan-lab:~/lab$ ls docs/final report.txt
ls: cannot access 'docs/final': No such file or directory
ls: cannot access 'report.txt': No such file or directory
student@okan-lab:~/lab$ ls "docs/final report.txt"
'docs/final report.txt'
student@okan-lab:~/lab$ ls docs/final\ report.txt
'docs/final report.txt'
student@okan-lab:~/lab$ cd docs/final report.txt
bash: cd: too many arguments
student@okan-lab:~/lab$ cd /root
bash: cd: /root: Permission denied
student@okan-lab:~/lab$ pwd
/home/student/lab
```

`command not found` A typing error, the wrong case (LS), or a program that is not installed (`sl`). On a normal Ubuntu the shell also suggests a package, e.g. “Command ’sl’ not found, but can be installed with: sudo apt install sl”.

`invalid option` The command does not have this option. The message itself suggests `ls --help`.

`No such file or directory` The name is wrong (remember case: `Docs` $\neq$ `docs`), or you are not in the directory you think you are in. Run `pwd` and `ls`, and use Tab completion.

`Not a directory` `cd` needs a directory; `README.txt` is a file.

Two errors for one name The shell splits a command line at spaces, so `docs/final report.txt` becomes *two* arguments, `docs/final` and `report.txt`. Put the name in quotes (`"docs/final report.txt"`) or escape the space with a backslash (`docs/final\ report.txt`). With `cd`, two arguments give `too many arguments`.

`Permission denied` You are not allowed in (`/root` belongs to the administrator). This is the protection system working as designed (Week 11). Do *not* react by putting `sudo` in front of commands.

`$'\r': command not found` A script with Windows line endings (Section 2.4).

A word on sudo

`sudo` runs one command as `root`, the administrator, after asking for *your* password. In this course you need it only to install software (`sudo apt install tree`). As `root`, the protection mechanisms of Weeks 2 and 11 no longer protect you from your own mistakes: one wrong command can damage the system. Never use `sudo` just because a command failed.

12 Summary

Week 9 in seven sentences

1. A Linux shell is available on the lab PCs, with WSL on Windows (`wsl --install`) or in a VirtualBox VM; the script `mcmp117_lab_setup.sh` creates the practice folder `~/lab`.
2. The terminal shows text, the shell (`bash`) interprets commands in user mode, and the kernel does the work through system calls.
3. For an external command the shell forks, the child execs the program file, and the shell waits (Week 3); builtins such as `cd` run inside the shell.
4. A command line has the form *command -options arguments*; words are separated by spaces and case matters.
5. `pwd` shows where you are; `ls` (with `-l -a -h -R -F`) shows what is there; `ls -l` displays the inode fields of Week 7.
6. Linux has one tree from `/`; `cd` moves with absolute or relative paths and the names `~ . .. -`.
7. `--help`, `man`, `help` and `type` explain commands; Tab, history and `Ctrl+C`/`Ctrl+L` save time.

13 Glossary

Absolute path A path starting at the root `/`; valid from any directory.

Builtin A command executed by the shell itself, without a new process (`cd`, `pwd`, `echo`, `history`).

Current working directory The directory from which a process resolves relative paths; part of each process's state.

FHS Filesystem Hierarchy Standard: which top-level directory holds what.

Hidden file A file whose name starts with a dot; shown only by `ls -a`.

Home directory A user's own directory, `/home/<user>`, abbreviated `~`.

Option A word starting with `-` or `--` that changes how a command works.

Prompt The text the shell prints when it is ready for a command.

Relative path A path starting from the current directory; may use `..`.

Shell The command interpreter: a user-mode system program that reads commands and runs them.

Terminal (emulator) The window program that passes keystrokes to the shell and displays its output.

WSL Windows Subsystem for Linux: runs a Linux distribution with a real Linux kernel inside Windows.

14 Lab Sheet

Re-create the lab folder first (`bash mcmp117_lab_setup.sh`), then start every exercise in `~/lab` unless stated otherwise. Write down the *commands* you used and the answers. Total: 100 points. Solutions are

in Section 16 — try first! In this lab you may use `cat file` to print a small file and `head -n N file` to print its first N lines.

Part A --- Getting started (13 points)

A1 **(4 pts)** Run `whoami`, `hostname`, `date` and `uname -r`. Write down the output and say in one sentence what each command reports.

A2 **(4 pts)** Run the lab setup script. What is the *absolute* path of your lab folder? Which command proves it?

A3 **(5 pts)** For each command line, name the command, the options and the arguments: (a) `ls -l -a docs`, (b) `ls -lh /var/log /tmp`, (c) `head -n 3 data/cities.txt`, (d) `cd`, (e) `ls --all`.

Part B --- Looking around (30 points)

B4 **(6 pts)** How many entries does `ls` show in `~/lab`? How many of them are directories? Which option makes the answer obvious?

B5 **(6 pts)** `docs` contains a hidden file. Find its name and print its contents.

B6 **(6 pts)** Which file in `docs` is the largest, and how many bytes does it have? Use the option of `ls` that sorts by size (find it in `ls --help`).

B7 **(6 pts)** Give the exact size of `logs/access.log` in bytes and the size shown by `ls -lh`. Show by calculation why they agree.

B8 **(6 pts)** How many files are in `photos`? How many of them end exactly in `.jpg` (lower case)? Which do not?

Part C --- Moving around (24 points)

C9 **(6 pts)** From `~/lab`, go to `projects/beta/docs` with *one* relative `cd` and print the current directory.

C10 **(6 pts)** From there, go to `projects/alpha/src` with one relative `cd`. Write the command and check with `pwd`.

C11 **(6 pts)** Starting in `~/lab/projects`, you type `cd alpha`, `cd ../beta/src`, `cd ..`, `cd -`. *Predict* the directory after each command, then check.

C12 **(6 pts)** Write two different commands that take you to `~/lab/src` *from any directory*. Why does `cd lab/src` work only in some directories?

Part D --- The system and help (18 points)

D13 **(6 pts)** Where is the program `ls` stored? What exactly is `/bin` on Ubuntu 24.04? Which file tells you the name and version of the distribution, and what does it say on your system?

D14 **(6 pts)** Print the PID of your shell and the lines `Name`, `State`, `Pid` and `PPid` of `/proc/<PID>/status`. Which data structure of Week 3 is this a view of, and why is the state S?

D15 **(6 pts)** Using `ls --help`, find the option that lists one name per line and the option that reverses the sort order. Use both together on `docs`. Then use `help cd` to explain what `cd -` does.

Part E --- Challenges (15 points)

E16 **(5 pts) Treasure hunt.** Start in `~/lab` and use only `cd`, `ls`, `pwd` and `cat`. (1) Read `README.txt`: go to the directory that holds "a small C program". (2) What does that program print? (3) With one relative `cd`, move to the directory of short text documents. (4) One file there has a space in its name; its last line is the password. (5) Finish in `projects/beta/src` with one relative `cd` and run `pwd`.

E17 **(5 pts) Link counts.** `ls -ld` shows link count 10 for `~/lab`, 4 for `projects` and for `projects/alpha`, and 2 for `docs`. Explain all four numbers with the Week 7 idea of hard links, and predict the link count of `projects/beta`.

E18 **(5 pts) Why a builtin?** Run `type cd`, `strace -e trace=chdir bash -c 'cd /tmp'` and then `pwd`. Explain, using the process concept of Week 3, why `cd` cannot be an ordinary program.

15 Review Questions

Conceptual

1. Explain the difference between a terminal, a shell and the kernel. Which of them run in user mode?
2. List the steps the shell takes when you type `ls docs`. Which system calls of Week 3 are involved?
3. What is the difference between a builtin and an external command? How can you tell which one a command is?
4. What is a hidden file? How do you list hidden files?
5. Explain each column of the line `drwxrwxr-x 4 student student 4096 Oct 12 08:49 projects`.
6. Why does Linux not need drive letters? What is `/proc`, and why is it called a virtual file system?
7. Explain the difference between an absolute and a relative path, with one example of each that names the same directory.
8. What do `~`, `.`, `..` and `-` mean for `cd`? Which of them are real directory entries?
9. Why does `ls docs/final report.txt` print two error messages? Give two ways to fix the command.

Numerical

10. A file has $3\,500\,000$ bytes. What does `ls -lh` show?
11. A file has $10\,300$ bytes. What does `ls -lh` show?
12. A directory has 5 subdirectories and 12 files. What is its link count?
13. A directory contains 7 files, each smaller than 4 KiB, and the file system uses 4 KiB blocks. What does the `total` line of `ls -l` show?
14. You are in `/home/student/lab/projects/alpha/docs`. Write the shortest relative path to `/home/student/lab/data`. How many levels up does it go?

Answers to the numerical questions

10. $3\,500\,000/2^{20} = 3\,500\,000/1\,048\,576 \approx 3.338$; one decimal, rounded up: `3.4M`.
11. $10\,300/1024 \approx 10.06$; at 10 or more no decimal is shown and the value is rounded up: `11K`.
12. $2 + 5 = 7$ (files do not count).
13. Each file occupies one block: $7 \times 4 = 28$, so `total 28`.
14. `../../../data`: up three levels (to `docs`' parent `alpha`, then `projects`, then `lab`), then down into `data`.

16 Lab Sheet Solutions

The outputs below are from `code/out_lab.txt` (script `code/demo_lab.sh`) and the other transcripts of this week. Dates, PIDs and system details will differ on your computer.

A1. See Section 4.3: `student`, `okan-lab`, the current date and time, and the kernel release (`6.18.44-fc-v114` on our machine). `whoami`: user name; `hostname`: computer name; `date`: date and time; `uname -r`: version of the Linux kernel.

A2. The script prints `Lab folder ready: /home/student/lab` (Section 2.4). `cd ~/lab` followed by `pwd` prints the absolute path `/home/student/lab` (with your own user name).

A3. (a) command `ls`, options `-l -a`, argument `docs`; (b) `ls`, options `-l -h` (combined), arguments `/var/log` and `/tmp`; (c) `head`, option `-n` with value `3`, argument `data/cities.txt`; (d) `cd`, no options, no arguments (goes home); (e) `ls`, long option `--all`, no arguments.

B4. 9 entries: 8 directories and the file `README.txt`. `ls -F` marks directories with `/` (`ls -l` shows `d`).

B5. `ls -a docs` reveals `.settings`; `cat docs/.settings` prints `editor=nano` and `color=on`.

B6. `-S` sorts by size, largest first: `ls -lS docs` puts `syllabus.txt` (578 bytes) at the top, followed by `linux_quotes.txt` (302) and `final report.txt` (53).

B7. `ls -l logs` gives $23\,604$ bytes; `ls -lh` shows `24K`, because $23\,604/1024 \approx 23.05$ and `ls -h` rounds up.

B8. 15 files. 12 end in `.jpg` (`img001.jpg` … `img012.jpg`); the other three are `IMG_014.JPG` (upper case — a different name on Linux), `holiday.jpeg` and `img013.png`. (Next week, wildcards such as `*.jpg` will count them for you.) `ls photos` shows all 15 names in three columns.

C9 and C10. `cd projects/beta/docs`, then `cd ../../alpha/src` (up to `beta`, up to `projects`, down to `alpha/src`); `pwd` prints `/home/student/lab/projects/beta/docs` and then `/home/student/lab/projects/alpha/src`.

C11. After `cd alpha`: `~/lab/projects/alpha`; after `cd ../beta/src`: `~/lab/projects/beta/src`; after `cd ..`: `~/lab/projects/beta`; `cd -` returns to `~/lab/projects/beta/src` and prints it:

C12. `cd /home/student/lab/src` (absolute; use your own user name) and `cd ~/lab/src` (the shell expands `~` to the absolute path of your home). `cd lab/src` is relative: it works only when the current directory is `/home/student`, the directory that contains `lab`.

D13. `type ls` → `/usr/bin/ls`; `ls -ld /bin` → a symbolic link to `usr/bin`; `/etc/os-release` names the distribution; on our machine `PRETTY_NAME="Ubuntu 24.04.4 LTS"` (Section 6).

D14. `echo $$`, then `head -n 7 /proc/$$/status` (Section 6; the PID differs every time). It is a view of the shell's *process control block* (Week 3): name, state, PID, parent PID. The state is S (sleeping) because, while `head` runs, the shell is blocked in `wait()` for its child.

D15. `-1` (one per line) and `-r` (reverse):

```
student@okan-lab:~/lab$ ls -1 -r docs
syllabus.txt
notes_03.txt
notes_02.txt
notes_01.txt
linux_quotes.txt
'final report.txt'
```

`help cd` says: *If DIR is "-", it is converted to $OLDPWD*, i.e. `cd -` goes to the previous directory (Section 8).

E16 (treasure hunt). (1) `README.txt` says `src/` holds "a small C program": `cd src`. (2) `cat hello.c`: the program prints `Hello from the lab!`. (3) `cd ../docs`. (4) `cat "final report.txt"`: the password is `This file name contains a space.` (5) `cd ../projects/beta/src`; `pwd` prints `/home/student/lab/projects/beta/src`.

E17. Link count of a directory $= 2 + s$ (s = number of subdirectories): its name in the parent, its own `.`, and one `..` in each subdirectory. `~/lab`: $s = 8$, $2 + 8 = 10$; `projects`: `alpha`, `beta`, $2 + 2 = 4$; `alpha`: `docs`, `src`, 4; `docs`: 2. Prediction for `projects/beta`: 4.

```
student@okan-lab:~/lab$ ls -ld ~/lab ~/lab/projects ~/lab/projects/alpha ~/lab/docs
drwxrwxr-x 10 student student 4096 Oct 12 08:49 /home/student/lab
drwxrwxr-x  2 student student 4096 Oct 12 08:49 /home/student/lab/docs
drwxrwxr-x  4 student student 4096 Oct 12 08:49 /home/student/lab/projects
drwxrwxr-x  4 student student 4096 Oct 12 08:49 /home/student/lab/projects/alpha
```

E18. See Section 7.4: `type cd` prints "cd is a shell builtin"; the child `bash` successfully calls `chdir("/tmp")`, but `pwd` afterwards still prints `/home/student/lab`. The current directory is per-process state; a child cannot change its parent's, so `cd` must run inside the shell process itself.

Grading hints. Give full points when the commands are correct even if dates, PIDs and system details differ. In C9–C12 and E16, deduct points if absolute paths are used where a relative path was required.

17 Looking Ahead: Week 10

Next week we stop just looking and start *changing* the file system in `~/lab`:

- creating and removing: `mkdir -p`, `touch`, `rm`, `rmdir` — and why `rm` has no recycle bin;
- copying and moving: `cp -r -i -v`, `mv`;
- wildcards `*`, `?`, `[abc]` and brace expansion `{a,b}`, `{1..5}` — expanded by the shell, not by the command (remember `echo` and `~` this week);
- viewing files: `cat`, `less`, `head`, `tail`, `wc`, `file`, `stat`, `du`, `df`;
- hard and symbolic links with `ln` and `ln -s` (inodes from Week 7).

Make sure you have a working terminal and the lab folder on your own computer before the lecture.

Further reading (optional)

- A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, 10th ed., Wiley: Section 2.2 (User and Operating-System Interface: command interpreters) and Section 13.3 (Directory Structure).
- R. H. and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*:
  - Chapter 5, "Interlude: Process API" (how a shell uses `fork`, `exec`, `wait`): `https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-api.pdf`
  - Chapter 39, "Interlude: Files and Directories": `https://pages.cs.wisc.edu/~remzi/OSTEP/file-intro.pdf`
- W. Shotts, *The Linux Command Line* (free book), Part 1, Chapters 1–5: `https://linuxcommand.org/tlcl.php`
- Microsoft, "How to install Linux on Windows with WSL": `https://learn.microsoft.com/en-us/windows/wsl/install`
- Filesystem Hierarchy Standard 3.0: `https://refspecs.linuxfoundation.org/FHS_3.0/fhs-3.0.html`